Requests Responses and JSON

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Verified Study Buddy

THE CONNECTED WEEK

Teach explicit success and rejection paths.

Guard field access, reject wrong types and assess a new specification with boundary cases.

Keep one habit: Ask what each gate establishes; zero, missing, rejected and unverified are different states.

MAIN LESSON · SECTION 1

Prepare this connected week

> Use one main plan and keep practice separate from the independent first attempt.

Learning goal and precision

Primary target: Parse **JSON** into Python records

Teach Week 26 as a two-session investigation using fictional data. A fictional activity service supplies **JSON** text. The **input contract** is a **JSON object** with name text and minutes an integer from 0 through 60.

The **output contract** will be a record with accepted and minutes fields. **Parsing** valid **JSON** is only the first step. Assess the three stated outcomes through the independent task, not by copying the worked result.

By the end, I can explain **request**s **response**s and **json** using a traced example and a stated limitation.

What to prepare

- Use the supplied scenario, student illustrations and blank paper. For numeric work, offer counters or a calculator after learners show the setup. Fictional records are supplied; personal data and paid accounts are unnecessary.
- Python 3 editor with a fresh local practice file
- Paper trace table and the student workbook
- Fictional **input** records from this lesson; no real student identifiers

Main teaching plan

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

Opening retrieval and readiness

Without opening the old lesson, explain one key idea from Tools and APIs. Give an example and a mistake that the idea helps you avoid.

Look for: Explain the **mechanism** and terminology of tools and api **contract**s

Reference example, not the only acceptable wording: Teacher explanation: Tools expose structured operations with **input** and **output contract**s.

Generating a tool call is a proposal; validation and authorisation decide whether it executes. For lookup_room accept exactly room_id as a nonempty string; return found plus capacity or a not-found result. It is read-only.

Reject unknown arguments and do not replace missing values with invented facts.

Without opening the old lesson, explain one key idea from User Need and AI Contribution. Give an example and a mistake that the idea helps you avoid.

Look for: Explain the **mechanism** and terminology of capstone requirements and corpus

Reference example, not the only acceptable wording: Teacher explanation: A grounded-assistant capstone begins with bounded requirements and a versioned permitted corpus.

A baseline can be a manual lookup **process** using the same **evidence**. Specify the user question, source authority, access, **output**/fallback **contract** and evaluation plan. Freeze corpus IDs and split cases before implementation.

Keep teacher material out of student retrieval.

Without opening the old lesson, explain one key idea from Verify Numbers and Dates. Give an example and a mistake that the idea helps you avoid.

Look for: Verify a numeric claim using its source, denominator and units

Reference example, not the only acceptable wording: Identify the source and population, retain units, and recompute with the correct denominator. Calendar validity does not establish the intended event date; preserve missing authority rather than guessing a replacement.

Reading obtains characters; validation checks whether their meaning and values satisfy rules.

▼ **Earlier wording, alternatives and picture transcripts**

Teach the learning cycle

Retrieval answer guidance

Tools and APIs

User Need and AI Contribution

Verify Numbers and Dates

Use the student module and workbook section with this exact week title. The worked case, independent question and answers below form one aligned lesson sequence.

What the learner must demonstrate

Request id=7 returns **status**=200 and {"id":8,"pages":90}. Explain why a successful **status** is insufficient and specify the handling.

Concept to explain

Week 22: **Request**s, **Response**s, and **JSON**

Term	Meaning
request	A message asking a system to return information or perform an action.
response	The information or status returned after a request.
JSON object	A collection of named fields and values written using JSON syntax, such as {"pages":80}.
status	A reported condition of a request or process, such as success, pending or error.

Prepare the class

This compact option is not the full-week schedule. Use the main teaching cycle and allow extra sessions for the investigation, independent work and project.

Allow 80 minutes: recall 8; explanation 12; worked demonstration 15; guided practice 20; independent assessment 15; project and reflection 10. Extend the lab into a second session when needed.

Preparation for the connected explanation

Readiness answer

End goal for the pictorial case

This added worked case illustrates the week’s main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

MAIN LESSON · SECTION 2

Teach parsing, schema checking and evidence as separate gates

> Keep activity, lunch and independent box contracts distinct.

Read the complete activity contract

A fictional activity service supplies **JSON** text. The **input contract** is a **JSON object** with name text and minutes an integer from 0 through 60. The **output contract** will be a record with accepted and minutes fields.

Parsing valid **JSON** is only the first step.

Current activity contract: exactly name and minutes. name is text with length greater than zero; this toy rule does not trim whitespace. minutes is an integer from 0 through 60; booleans and decimal-number values are rejected. No extra or duplicate fields are allowed. The output always has accepted and minutes; any rejection returns false and null.

Duplicate rejection is an explicit classroom policy; JSON libraries may otherwise retain a repeated name’s last value. Non-JSON NaN/Infinity constants are rejected. These checks are for short supplied classroom cards, not a production network parser.

Explain the original illustration’s top-level limitation

The **output** is {"accepted": true, "minutes": 12}. **JSON** uses lowercase true, false and null; Python uses True, False and None. **json**.loads converts between representations.

The worked **output** line assumes the parsed top-level value is a dictionary; a robust **interface** must reject other top-level types before accessing fields.

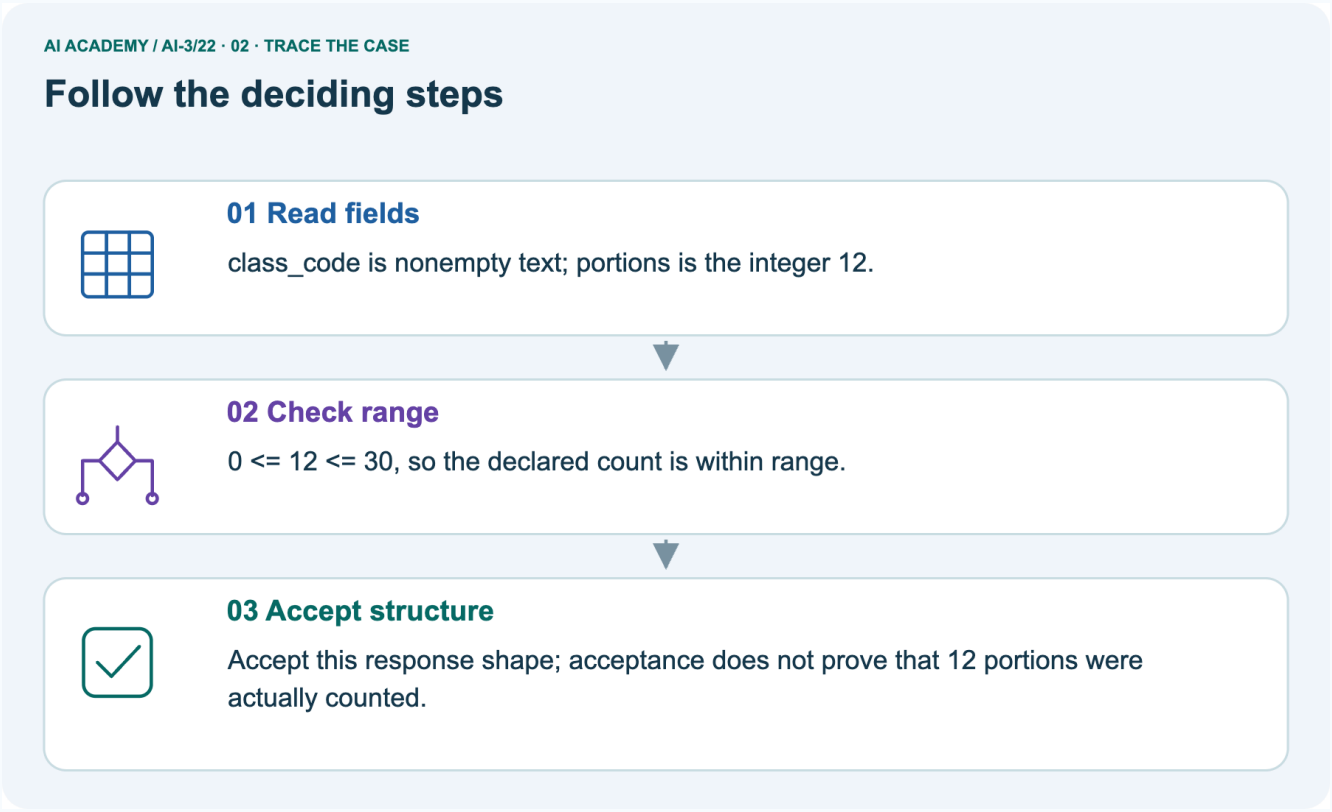
Read the lunch-response scope

A lunch-count **response** must satisfy a declared **JSON contract**.

The toy **response** has exactly class_code text and portions integer 0..30. Example: {"class_code":"R8","portions": 12}.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Trace the accepted structure



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Read fields	class_code is nonempty text; portions is the integer 12.
2. Check range	0 <= 12 <= 30, so the declared count is within range.
3. Accept structure	Accept this response shape; acceptance does not prove that 12 portions were actually counted.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Read the picture in order. Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. The arrows show which recorded **input** or intermediate result each decision uses.

Explain the deciding step aloud before checking the changed case; pointing to the final answer alone does not show how it follows.

Predict quoted portions

Change: Replace 12 with the text "12".

Prediction: Reject the **response** under this strict **contract**.

Reason: Digits inside quotes represent text, whereas portions requires an integer.

State the real-world evidence limit

JSON well-formedness, schema validity and real-world truth are different checks.

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

▼ Teacher implementation reference

Python reference: json.loads decodes JSON text; JSONDecodeError reports malformed documents; object_pairs_hook permits duplicate-name handling and parse_constant can reject NaN/Infinity extensions. The lesson uses these longstanding interfaces, not new version-specific features. Official documentation: https://docs.python.org/3/library/json.html

▼ Additional practice · Requested and returned ID

Request is for B7; **response** is {"id":"B7","pages":120}. Another **response** has malformed **JSON**.

What checks precede use?

Request success, **JSON** validity, matching id, required fields and numeric type. Handle malformed **response**s explicitly.

▼ Earlier wording, alternatives and picture transcripts

Explain the mechanism

Input → **Process** → **Output**

Trace the information. At each stage, ask what changed and how you could test it.

Which stage could fail even when the other two are correct? Explain:

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.

1. **Request** requires question:text and source_ids:list.

2. **Response** requires answer:text, citations:list, **status**:text.

3. Missing citations or wrong source_ids type fails validation.

4. Error **response** should state code, message, and safe next step.

Recalculate, retrace, or restate the reasoning in your own representation:

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

Concrete teaching case

Exact explanation to model

Explain the added worked example

Key terms

Term	Meaning
JSON	A text format for structured values
Parsing	Converting formatted text into program values
Interface	The agreed way one part of a system exchanges data with another
Contract	Specified inputs, outputs and error behavior

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

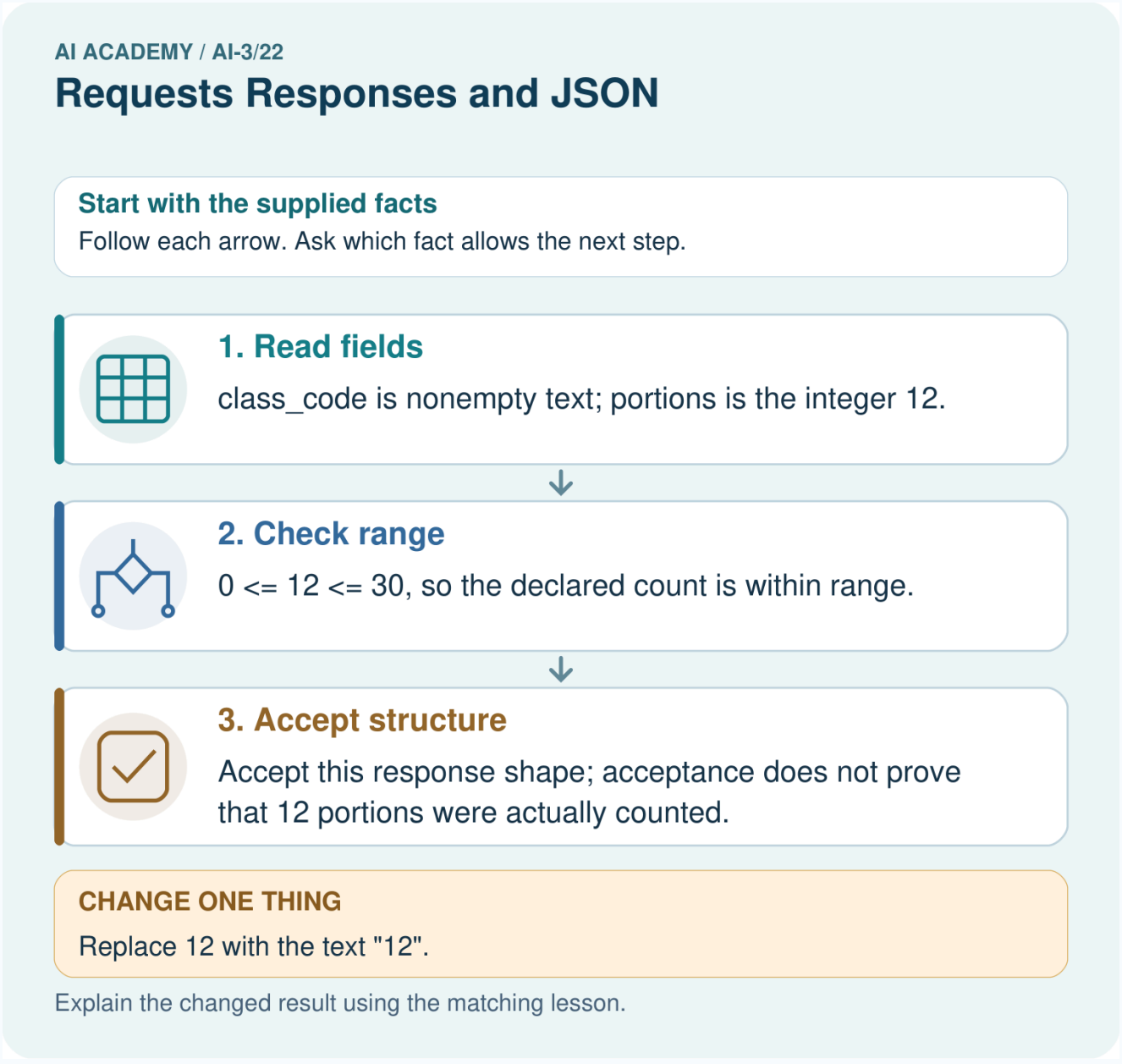
> **JSON text contains {"name": "Sketch", "minutes": "12"}. Does parsing fail, or does validation fail?**

Thinking move	Teacher prompt
Plan	What is given? What is the question? Which procedure fits these facts?
Do	Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.
Check	Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: Requests Responses and JSON

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing in the pictured case

Ask while showing the picture

MAIN LESSON · SECTION 3

Guide the explanation, then remove help

> Collect each learner’s reasoning before showing feedback.

Main completion prompt

Completion task: Classify {"minutes": "5"} and {minutes: 5} as valid **JSON** text or malformed **JSON**.

Expected reasoning and fading help

Teacher check: The first is valid **JSON** but its minutes field fails the integer schema. The second is malformed because the key is not double-quoted.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a **response** from every learner before discussion.

Individual misconception check

Use the worked case for **JSON** and **interface contract**s. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: An **interface** should state what happens for malformed text, wrong structure and invalid values. Do not expose raw private **input** in error messages. Use only the fictional records supplied for practice.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial question and changed-case reasoning

Would converting the text silently prove the source count is correct? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

No. Under the stated strict **contract** reject the wrong type. Any explicit conversion policy would change the **contract**, and factual verification remains separate.

Changed-case check: Reject the **response** under this strict **contract**. Digits inside quotes represent text, whereas portions requires an integer.

Changed-case reasoning: Reject the **response** under this strict **contract**. Digits inside quotes represent text, whereas portions requires an integer.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

Keep each learner explaining

- Ask: Which facts are given, and which would we have to assume? Learners mark relevant details before discussing answers.
- Say: Predict first. Show the rule, information path or calculation that would produce the result. A correct guess is not yet an explanation.
- In pairs, one learner solves while the other checks **evidence**; exchange roles. Remove the worked answer before independent assessment.

▼ Earlier wording, alternatives and picture transcripts

Compact alternative: this two-session sequence covers selected core tasks. Use the three-session weekly plan for the full lesson, independent transfer, feedback and project work; allow extra time for extensions.

Demonstrate and guide practice

Board sequence

Write the given facts: **Request** is for B7; **response** is {"id":"B7","pages":120}. Another **response** has malformed **JSON**.

Write the question to explain: What checks precede use?

Reveal after **prediction**s: **Request** success, **JSON** validity, matching id, required fields and numeric type. Handle malformed **response**s explicitly.

Use the earlier changed question as optional later practice. Reserve Specify the new label/count interface independently for the first independent attempt before hints or feedback.

Classroom dialogue

Reveal reasoning one step at a time. Explain why a tempting alternative fails. Write units and denominators for arithmetic, trace conditions for decisions, and identify supporting sources for claims.

Model these guided questions

Parsing succeeds; validation fails because minutes is text. Exit: No. Syntax, schema validity and **observation**al truth are separate checks.

Case-specific teaching sequence

Stage	Teacher action
Session 1 start to 7 minutes	Retrieve the prerequisite and explain the purpose in the opening section.
7 to 20 minutes	Read the worked example and diagram; ask learners to trace each step aloud.
20 to 33 minutes	Complete W1 and W2, then compare predictions with observed results.
33 to 45 minutes	Collect the guided prediction before running; discuss a mismatch and its cause.
Session 2 start to 8 minutes	Retrieve the main rule without showing the previous answer.
8 to 20 minutes	Use W3 to diagnose and repair the stated failure; rerun an earlier case.

Stage	Teacher action
20 to 35 minutes	Collect W4 independently, including code or plan, predictions and explanations, before hints.
35 to 45 minutes	Discuss W5 and the exit; use the fresh retry when support was needed.

Reduce help deliberately

Individual reasoning check

Pictorial practice question

Reasoned teacher answer

MAIN LESSON · SECTION 4

Run the supplied investigation

> Keep expected results, observed paper traces and remaining limits distinguishable.

Supplied materials and answer guidance

Eight-card first outcomes: accepted 12; syntax; non-object; missing field; string minutes; boolean minutes; out-of-range 61; accepted 0. Every rejection returns accepted=false and minutes=null. W2 first string is valid JSON but minutes is text and name is absent under the full activity contract; second has malformed unquoted key. W3 checks dict before .get. W1 null→None and True→true.

Independent label/count needs nonempty text, integer 0..10, missing-field rejection and a stated extra-field policy. No accepted record proves a physical count.

Run and record

1. Predict → Test → Record → Explain

2. Keep expected and actual results separate. Surprises are useful data when recorded honestly.

3. Use the eight supplied activity JSON cards and complete guarded validator. Separate syntax, top-level structure, exact fields, types and range errors; preserve a real zero and return null on rejection. Use paper traces or recorded Python execution, not invented service responses.

4. What did the hardest test teach you that the normal test did not?

Keep an inspectable record

Record	What to capture
Input or evidence	Exact data, instruction or claim
Expected result	Answer or safe fallback declared before the run
Observed result	Actual result, including errors
Explanation	Rule, calculation or evidence explaining the outcome
Next test	One justified change and a new prediction

Diagnose the first misunderstanding

A successful run proves the conclusion: An **interface** should state what happens for malformed text, wrong structure and invalid values. Do not expose raw private **input** in error messages.

Use only the fictional records supplied for practice.

Matching the example is enough: Require the new W4 case, explicit **prediction**s and an explanation. Fresh retry: label Tray and score integer 0 through 5.

Cases score 0 accepted, score 6 rejected, score null rejected; **JSON** list rejected before field access.

A convenient shortcut preserves meaning: Single quotes around property names are not valid **JSON** syntax. Use double quotes in the **JSON** text.

If the whole parsed value is a list, do not call .get on it: check the top-level type first and report a schema failure.

Choose support from the response

What the response shows	Next teaching action
Missing or misread facts	Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.
Wrong procedure	Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.
Arithmetic or implementation error	Trace intermediate values or lines; preserve the first mismatch and retry that step.
Correct result without a reason	Ask for a trace and a changed case before marking independent understanding.
An unsupported wider claim	Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

▼ **Earlier wording, alternatives and picture transcripts**

Run the weekly evidence lab

Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.

Make the lab runnable

Use the supplied investigation for shared practice. Keep predictions and observed paper-trace results separate. Reserve Specify the new label/count interface independently for the individual first attempt before feedback; do not use the reserved case as a paired lab task.

Diagnose and reteach

For guessing, return to given facts and **request** a trace. For vocabulary without application, use one example and one non-example. For arithmetic errors, check the setup before the calculation.

Finish with a different uncoached case before recording mastery.

Address these additional misconceptions

Match the support to the first error

MAIN LESSON · SECTION 5

Assess the selected label/count specification

> Reveal this key after the first attempt; do not assess by whether the program merely ran.

Selected independent question and key

Selected case: Define a **JSON contract** for {"label": "Box", "count": 4}: nonempty text label, integer count 0 through 10. Give a valid case, a wrong-type case and a missing-field case with expected acceptance.

Valid Box/4 accepted; count "4" rejected for type; absent count rejected as missing. State that malformed **JSON** and non-object top levels are also rejected.

Judge trace and scope

Dimension	Evidence required
Explanation	Names the applicable idea or procedure and ties it to the supplied facts.
Trace and result	Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.
Limits	States the assumptions, missing evidence or conditions under which the result would change.

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.

Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

Give one repair action

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

Current guidance

Current activity guidance: retain every rejection stage and output from the supplied table. W2's minimal snippets isolate JSON syntax versus integer type; neither is a complete valid activity record because name is absent. W3 requires a top-level dictionary before field access. W5 distinguishes parsing, schema validation and source truth.

The published guarded program handles non-object JSON without crashing and preserves valid zero.

▼ Workbook questions and reasoned explanations

W1 Translate a value

What Python value does **JSON** null become? What **JSON** value represents Python True?

None and true, respectively.

Parsing JSON null produces Python None, while serialising Python True produces **JSON** true. The spelling belongs to the representation: lowercase true is **JSON** text, not the Python Boolean name True.

Keep **parsing** and serialisation directions clear.

W2 Separate failures

Classify {"minutes": "5"} and {minutes: 5} as valid **JSON** text or malformed **JSON**.

The first is valid **JSON** but its minutes field fails the integer schema. The second is malformed because the key is not double-quoted.

W3 Repair the interface

A parser accepts a **JSON** list then crashes on .get. What check belongs before field access?

Require type(record) is dict and handle failure before any .get or keyed field access.

W4 Specify independently

Define a **JSON contract** for {"label": "Box", "count": 4}: nonempty text label, integer count 0 through 10. Give a valid case, a wrong-type case and a missing-field case with expected acceptance.

Valid Box/4 accepted; count "4" rejected for type; absent count rejected as missing. State that malformed **JSON** and non-object top levels are also rejected.

W5 Limit parsing

A **JSON** record parses successfully. Does that prove its **observation** is correct?

No. Syntax, schema validity and **observation**al truth are separate checks.

No. **Parsing** checks **JSON** syntax. The schema separately checks the expected fields, types and ranges.

Collection **evidence** is a third question: even a well-formed, schema-valid duration could have been measured or recorded incorrectly.

▼ Earlier wording, alternatives and picture transcripts

Assess and explain the answers

Worked case answer

Request success, **JSON** validity, matching id, required fields and numeric type. Handle malformed **response**s explicitly.

Additional transfer practice question

An API returns {"ok":false,"error":"not_found"}. Your app tries to display **response**.title. What should it do instead?

Reasoned practice answer

Check the success or error fields before using result data. Show a useful not-found **response** and allow correction of the **request**.

What counts as evidence

Criterion	0 points	1 point	2 points
Result	Incorrect or absent	Partly correct	Correct with necessary conditions
Reasoning	No supporting trace	Some steps	Clear mechanism, calculation or source support
Limit	Overclaims certainty	Vague limitation	Relevant uncertainty or missing fact
Transfer	Copies without adapting	Adapts with prompts	Solves a changed case independently

Use 6/8 as a classroom checkpoint, with 2 required for both Result and Reasoning. This is a proposed teaching rule, not a validated certification threshold.

Do not count a coached answer as independent mastery; correct unsafe or unsupported claims regardless of total.

Feedback

Ask the learner to find the **evidence** that challenges this claim: “Every API **response** contains successful result fields.” Keep the first attempt and an uncoached retry separate.

Original transfer answer

The **response** is structurally plausible but for the wrong ID. Check **status**, parseability, required fields, types and matching ID before use.

Reject or report the mismatch; do not silently attach another book's page count to id=7.

Explanations for the additional workbook examples

Assess independent transfer

Selected case answer

Feedback that the learner uses

MAIN LESSON · SECTION 6

Connect, adapt and return

> Retain the project evidence and revisit the reasoning on a changed case.

Project evidence to keep

Use a stable **request/response contract** between **interface**, retriever, model, and verifier.

Keep the decision, **input**, result, explanation and remaining limitation in the project **evidence** record. Retrieve this record next week rather than restarting the project.

For your Verified Study Buddy, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

Support while preserving the reasoning goal

Provide one short approved passage; separate instruction, **input**, **output** and **evidence**. Stretch: introduce an unsupported claim or conflicting source and validate claim by claim with a fallback.

Color **JSON** punctuation separately from values. Extension: serialize a valid result, parse it again and check the data values are preserved.

Independent W4 **evidence**: Valid Box/4 accepted; count "4" rejected for type; absent count rejected as missing. State that malformed **JSON** and non-object top levels are also rejected.

Record whether the learner needed no help, a hint or explanation. Fresh retry: label Tray and score integer 0 through 5. Cases score 0 accepted, score 6 rejected, score null rejected; **JSON** list rejected before field access.

Plan the delayed check

Suggested return points in this level: ai-3/23, ai-3/25, ai-3/28. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent **response** and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

Record	Teacher evidence
Learner code and date	
Concept and case used; prior exposure	
Explanation / trace / limits	
Support and first repair	
Changed case and delayed outcome	
Next teaching action	

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Bridge to next week

Ask for one decision the learner can now justify and one situation requiring more **evidence** or human review. Confidence ratings are separate from correct independent performance.

Next: Week 23: Block-to-Python Bridge. Retrieve today's **evidence** and identify what carries forward and what is new.

Next is Week 23: Block to Python Bridge. Carry forward explicit input/output **contract**s and guarded error paths when translating a visual procedure into code.

▼ **Earlier wording, alternatives and picture transcripts**

Connect and extend learning

Verified Study Buddy

Support and stretch

Exit reflection

Bridge to the next week

Support and fresh reassessment

Plan the delayed return

Project evidence from this case

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference: alternative planning checklist**

REFERENCE · SECTION 7

Reference: alternative planning checklist

> Use as a reminder alongside the main route rather than teaching the cycle twice.

Preparation and schedule

Teaching target: Parse **JSON** into Python records

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual **response**s. No paid AI account is required for the paper reasoning activities.

Session · Plan · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Plan:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Plan:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Plan:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

Practice and assessment reminders

Before class, solve the supported practice: Classify {"minutes": "5"} and {minutes: 5} as valid **JSON** text or malformed **JSON**.

Reasoned practice answer: The first is valid **JSON** but its minutes field fails the integer schema. The second is malformed because the key is not double-quoted.

Check every learner before discussion: Use the worked case for **JSON** and **interface contract**s. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Misconception repair: An **interface** should state what happens for malformed text, wrong structure and invalid values. Do not expose raw private **input** in error messages. Use only the fictional records supplied for practice.

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

Extension: Replace 12 with the text "12". Require a **prediction** and a reason before comparing. Expected result: Reject the **response** under this strict **contract**.

Reason: Digits inside quotes represent text, whereas portions requires an integer.

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

Criterion	Not yet	With support	Independent
Explanation	Names an answer without a deciding fact	Uses a hint to link fact and decision	Explains the deciding fact in own words
Trace or test	Cannot connect input and result	Completes steps with prompts	Traces or tests a changed case accurately
Limits and revision	Claims more than evidence supports	Names a limit after a prompt	States a defensible limit and useful next test

Project checkpoint: Evaluate and revise. Compare with your starting method, explain errors and record a justified revision.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and **evidence**.

▼ **Reference: feedback and records**

REFERENCE · SECTION 8

Reference: feedback and records

> Use digital tools where they help retain an explanation and its evidence.

Keep practice and independent evidence separate

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Repair and portfolio records

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-boundary and indexing lessons deliberately include failure cases.
</details>

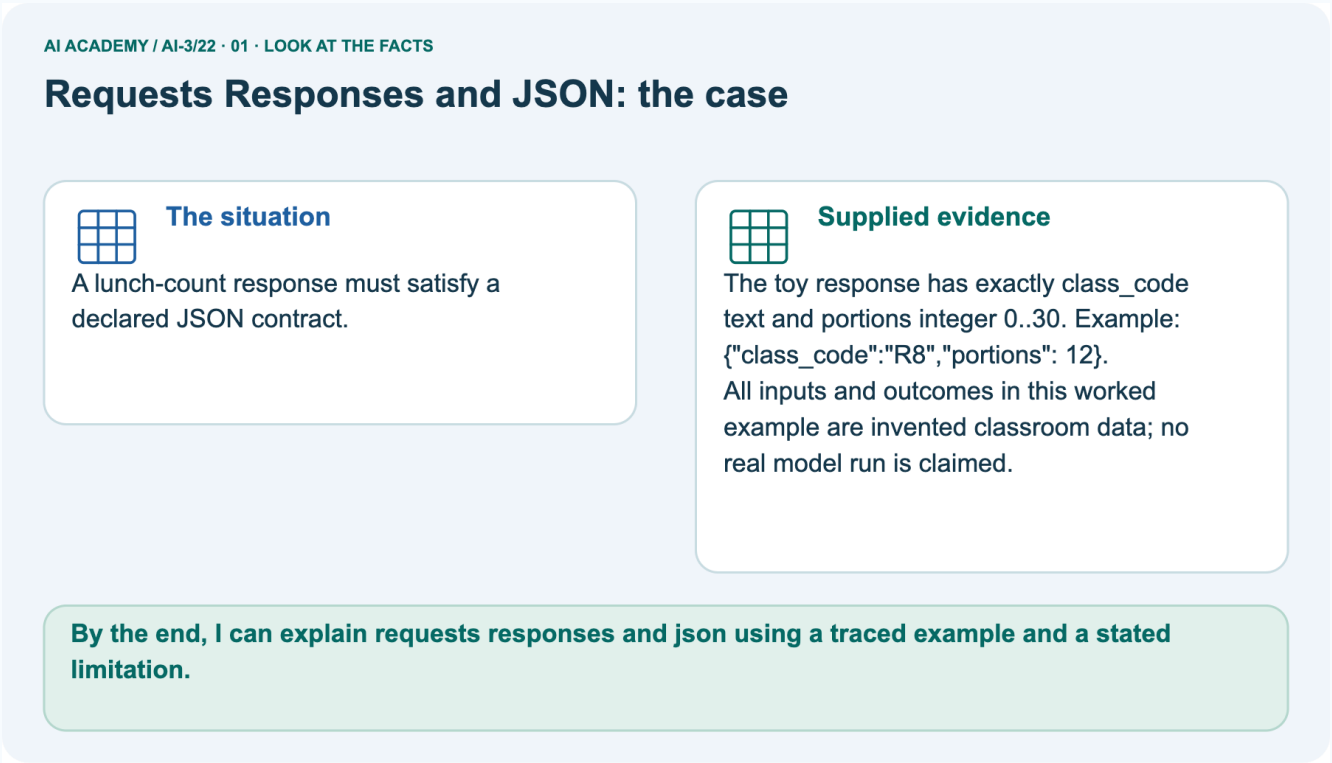
<details>
<summary>Reference: teach with the six pictures</summary>
</details>

REFERENCE · SECTION 9

Reference: teach with the six pictures

> Choose the picture for the teaching move and connect it to the explanation.

01. Requests Responses and JSON: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

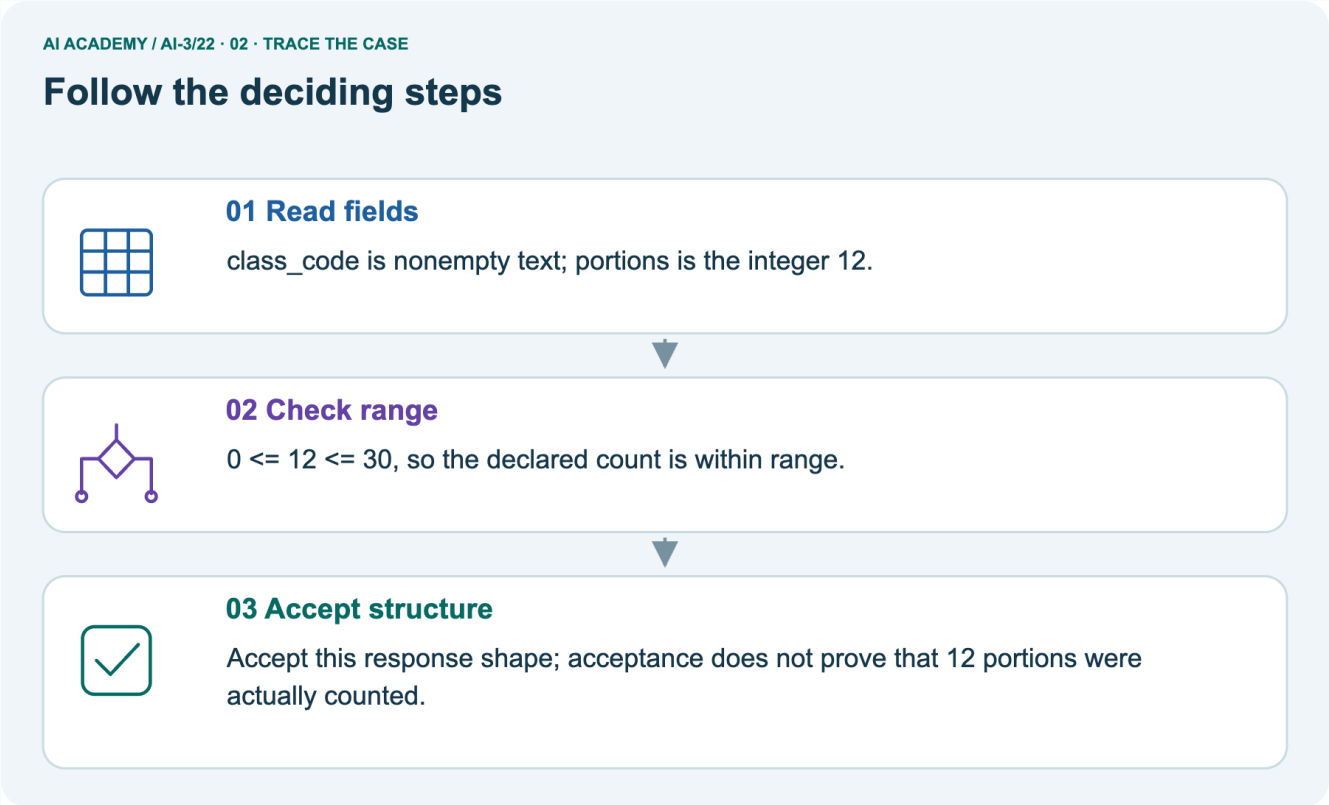
A lunch-count **response** must satisfy a declared **JSON contract**. Supplied facts: The toy **response** has exactly class_code text and portions integer 0..30. Example: {"class_code":"R8","portions": 12}.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

02. Follow the deciding steps



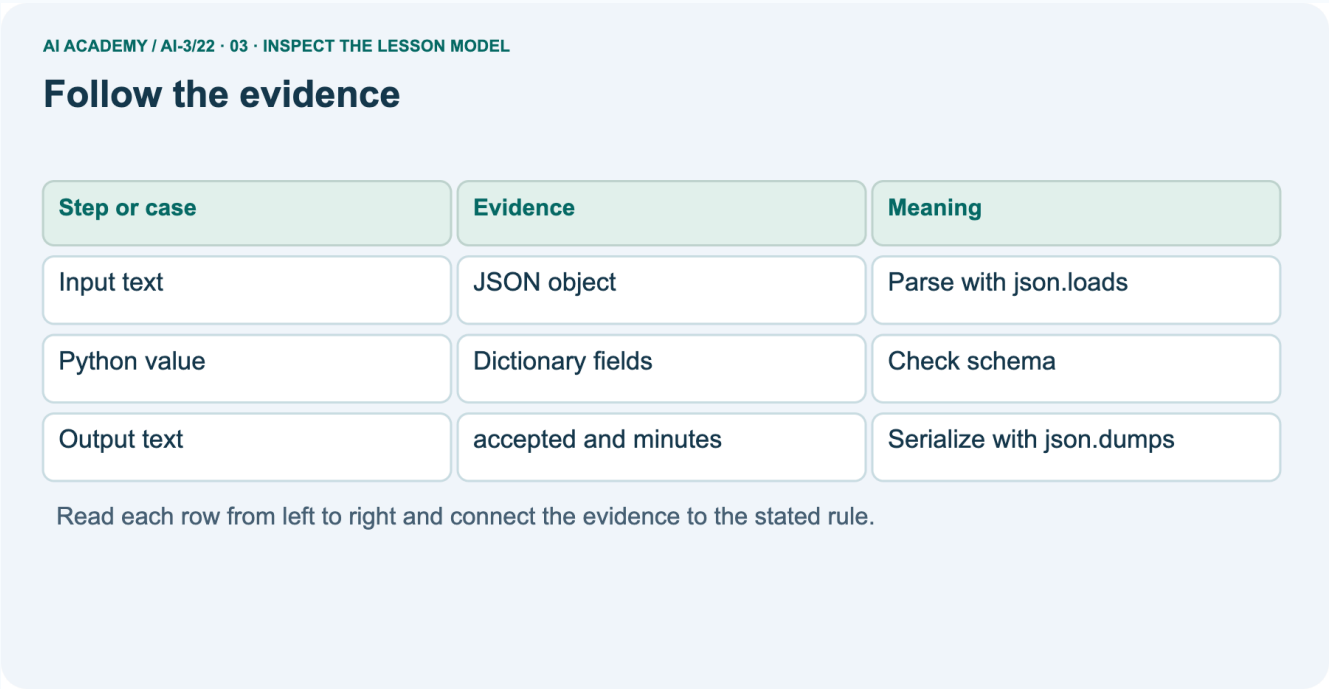
Follow the labelled picture with the matching explanation. Open the vector diagram

Read fields: class_code is nonempty text; portions is the integer 12. → Check range: 0 <= 12 <= 30, so the declared count is within range. → Accept structure: Accept this **response** shape; acceptance does not prove that 12 portions were actually counted.

Point to each numbered stage and explain which recorded **input** it uses.

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

03. Follow the evidence



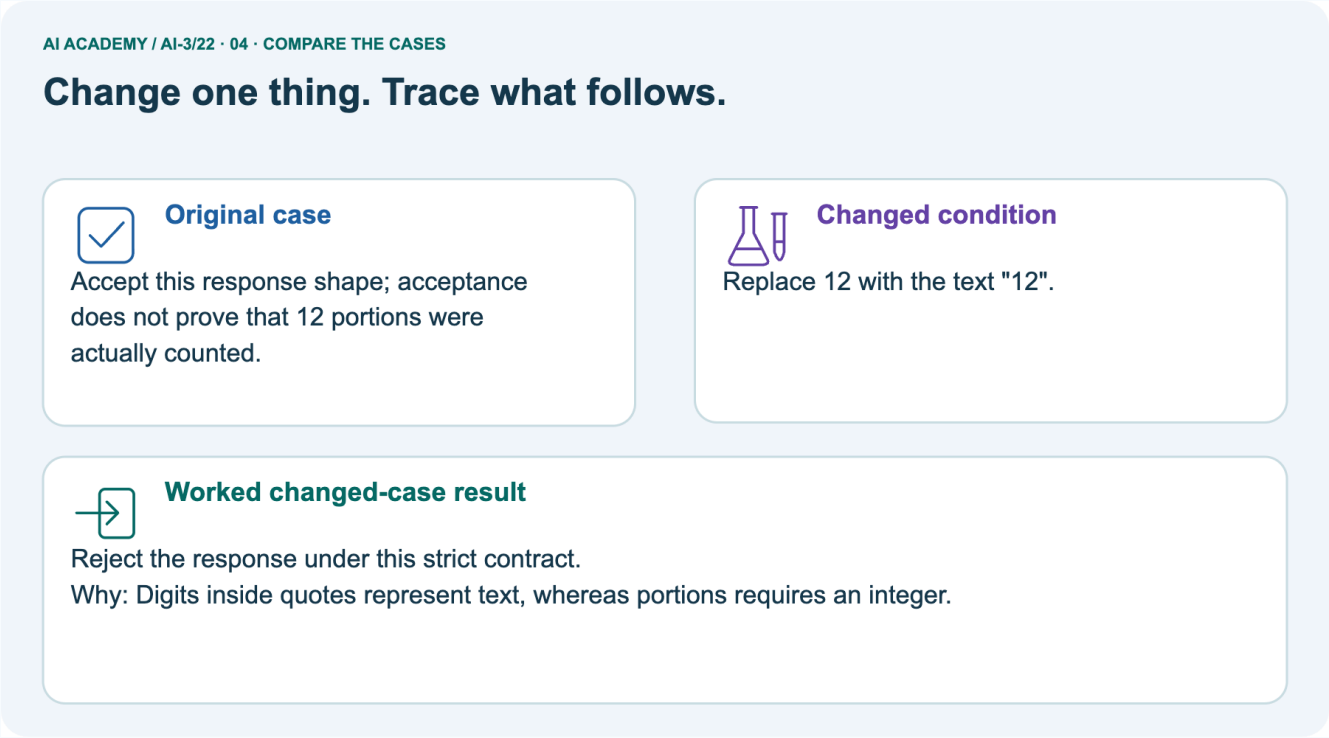
Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. **Input** text | **JSON object** | Parse with **json**.loads. Python value | Dictionary fields | Check schema. **Output** text | accepted and minutes | Serialize with **json**.dumps

Read the labels and values. Explain the relationship the model shows.

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. Changed condition: Replace 12 with the text "12". Expected result: Reject the **response** under this strict **contract**.

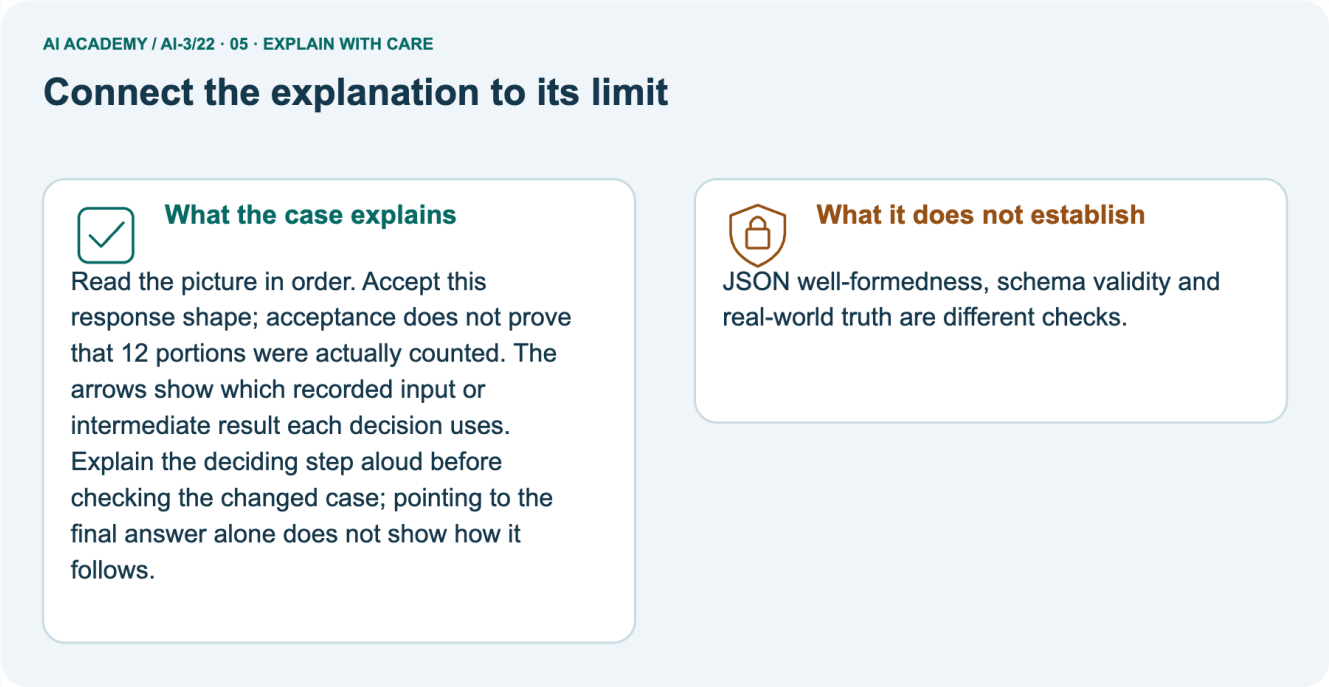
Reason: Digits inside quotes represent text, whereas portions requires an integer.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: Digits inside quotes represent text, whereas portions requires an integer.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture in order. Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. The arrows show which recorded **input** or intermediate result each decision uses.

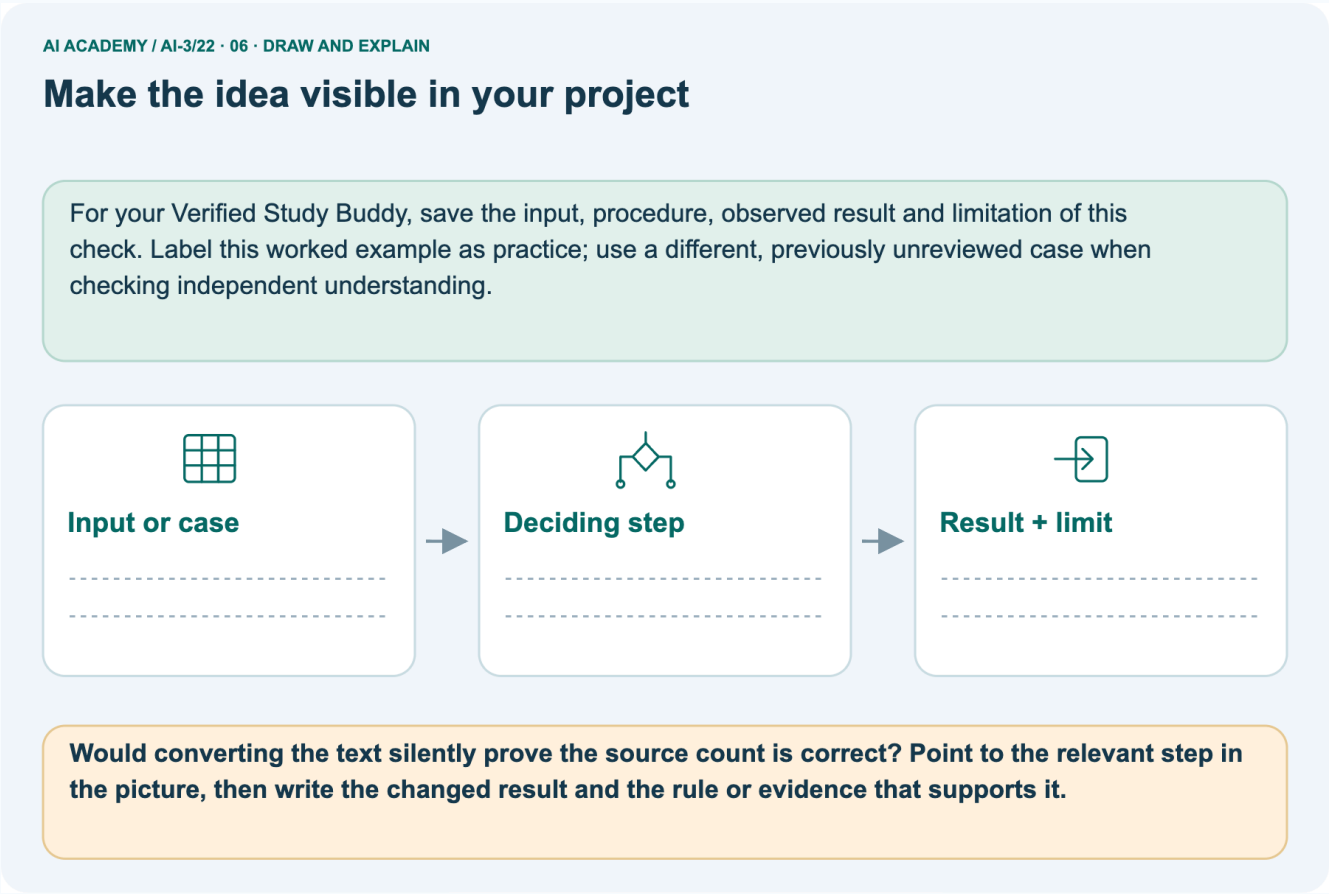
Explain the deciding step aloud before checking the changed case; pointing to the final answer alone does not show how it follows. Limit: **JSON** well-formedness, schema validity and real-world truth are different checks.

Explain the case, then point to a limit on the claim.

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: **JSON** well-formedness, schema validity and real-world truth are different checks.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Verified Study Buddy, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: Would converting the text silently prove the source count is correct? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Supported practice key: No. Under the stated strict **contract** reject the wrong type. Any explicit conversion policy would change the **contract**, and factual verification remains separate.

Changed-case check: Reject the **response** under this strict **contract**. Digits inside quotes represent text, whereas portions requires an integer.

▼ Earlier wording, alternatives and picture transcripts

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

01. Requests Responses and JSON: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.